

Individual Reflection — Antonio

My Contributions

Whisper Optimization

I profiled the full pipeline to find where time was being spent. The assumption going in was that the LLM call would be the slow part — in practice, Claude Haiku returns in a few seconds, while Whisper on a 20-minute audio file was taking close to 8 minutes with the default configuration. I implemented four optimizations, all of them zero-cost:

Switched from the multilingual base model to base.en: The English-only variant is faster and more accurate for English audio.

Set beam_size=1 (greedy decoding): Removes beam search overhead (~2× faster) with negligible accuracy loss for our task.

Set cpu_threads to use all available cores: Transcription runs fully parallel.

Passed language='en' explicitly: Skips the automatic language-detection pass.

Result: Processing time dropped from 7 min 50 s to 2 min 12 s on the same 20-minute recording — a ~3.5× speedup with near-identical transcript quality.

SummarAI

Individual Reflections

Evaluation Script

I built `data/eval/evaluate.py`. ~~The script loads all transcripts from `eval_set.jsonl`, calls the same `analyze()` function used in production, compares predicted action items against gold labels using the team's agreed matching rule, and outputs a JSON file with aggregated precision, recall, and F1 to `data/eval/results.json`.~~ The matching logic uses normalized text similarity with a threshold of 0.6 and greedy one-to-one assignment so each gold item is matched at most once.

Antonio · Bojana · Jo · Martí · Smaragda

NLP Group Project · 2025–2026

Installation Guide

I wrote `docs/installation_guide.md` covering Python version requirements, virtual environment setup, ffmpeg installation on macOS, Linux, and Windows, `.env` configuration, and the exact commands to run the server.

Labeled Transcripts

I contributed approximately 8 labeled transcripts to `eval_set.jsonl`, including sprint planning sessions and one deliberately ambiguous case where responsibility is implied but no specific person is named.

What I Learned

Working on this project gave me a much more concrete understanding of how ASR models like Whisper actually work under the hood — specifically the trade-off between multilingual and language-specific models, and what beam search is doing in a seq2seq decoder. Before this I understood beam search as a concept from the course; seeing it actually cost us 3.5× in latency with no measurable quality benefit made the accuracy-efficiency trade-off feel real rather than theoretical. I also developed a much clearer

sense of what precision and recall mean as evaluation metrics for an information extraction task. A single accuracy number would have hidden the difference between the model missing real action items (recall failures) and inventing ones that were not there (precision failures) — two completely different failure modes with different implications for trust.

Individual Reflection — Marti

My Contributions

Frontend

I built the entire frontend as a single HTML file (`src/static/index.html`) with no framework dependencies — plain JavaScript and CSS only. The interface includes a drag-and-drop upload area, a results panel with the summary, color-coded takeaway bubbles (purple for decisions, grey for notes), an action item checklist, and a collapsible sidebar showing the timeline of saved summarizations per project. The layout is fully responsive and works on iPhone screens.

Loading state: Transcription can take several minutes, so the interface shows a spinner with a running elapsed timer. Results render progressively as the JSON comes back from the API.

Error handling: All failure states are handled explicitly — unsupported file type, missing API key, model error — so the user gets a clear message rather than a silent failure.

Failure Mode Analysis

After the evaluation was run, I documented the most informative failure cases in `data/eval/failure_analysis.md`. Each case includes the transcript excerpt, the model's output, the gold label, and a hypothesis for why it failed. Main patterns:

Whisper mishearing proper nouns: Causes the owner field to be wrong even when the task is correct.

Phantom action items: The model generates commitments from hypothetical discussions.

Zero output on non-meeting audio: Correct behavior for podcasts, but may confuse users.

Implied responsibility: When no person is named, the model leaves owner null rather than guessing.

Labeled Transcripts

I contributed approximately 8 labeled transcripts to `eval_set.jsonl`, deliberately designing difficult cases: meetings where no one is explicitly assigned a task, shared responsibilities, and informally stated deadlines.

What I Learned

The most important thing I took away is how NLP errors propagate through a pipeline. Whisper's transcription errors do not stay in the transcription layer — they flow directly into the LLM output. If Whisper mishears a name, the action item will have the wrong owner even if the model does everything else correctly. This cascading failure is hard to see from an aggregate F1 score alone, and it made me understand why failure mode analysis is treated as a required component of the assignment. I also learned what information extraction means as an NLP task — it sits between pure classification and open-ended generation, and the challenge is that the output structure is constrained but the surface form is open, which is exactly why simple rule-based approaches fail on real, messy transcripts.

Individual Reflection — Bojana

My Contributions

Backend and NLP Pipeline

I built the entire server-side application (`src/app.py`). This includes the FastAPI server, the file upload endpoint, and the full two-stage NLP pipeline: routing audio files through `faster-whisper` for transcription, then passing the transcript to Claude Haiku 4.5 to extract the summary, typed takeaways, and action items. I also built all project management endpoints — creating, listing, and deleting projects and saved summarizations — using a thin HTTP wrapper to avoid adding a full Supabase client library as a dependency.

Input type routing: Audio and video files need to be written to a temporary file on disk before Whisper can process them. Text files are read directly into memory. Temporary files are cleaned up reliably using `try/finally` even when transcription fails.

System Prompt and Output Schema

I designed the system prompt and the JSON output schema that enforces the structure Claude must return. The schema defines three required fields — `summary`, `key_takeaways`, and `action_items` — with strict types and no additional properties allowed. Early versions produced inconsistent output: action items appearing inside the summary, takeaways missing their type tag, or the model inventing owners not in the transcript. I iterated on the prompt until the model reliably separated the three output types and only included owners and deadlines when explicitly stated. The final prompt and schema are saved in `prompts/`.

Labeled Transcripts

I created approximately 8 transcripts for `eval_set.jsonl`, including standup meetings, sprint planning sessions, and one edge case where a speaker discusses a hypothetical future action without committing — designed to test whether the model invents phantom action items.

What I Learned

This project gave me a real understanding of prompt engineering as an NLP technique rather than just a trick. When we started, the model would sometimes invent owners not mentioned in the transcript, or merge two separate action items into one. Fixing this required understanding why the model was doing it — it was following conversational implicature, inferring responsibility from context the way a human would — and then writing a prompt constraint specific enough to override that default behavior. I also came to appreciate what makes LLM-based information extraction different from traditional approaches: the model handles paraphrase, messy grammar, and implicit references naturally. The cost is that you cannot fully predict or control the output, which is exactly why structured schemas and evaluation against gold labels are necessary.

Individual Reflection — Jo

My Contributions

Supabase Database Layer

I designed and set up the entire persistence layer for the app. I wrote `supabase_schema.sql`, which defines two tables: `projects` (for grouping meetings) and `summarizations` (for storing the full structured output). A key design decision was storing `key_takeaways` and `action_items` as JSONB columns rather than flat text, preserving the structured data as queryable fields rather than serialised strings.

Cascade delete: The schema includes a foreign key from `summarizations` to `projects` with cascade delete, so removing a project automatically removes all its meetings.

Row-level security: RLS policies allow public read/write access without requiring user accounts — appropriate for a POC and documented as a limitation for production.

I also verified that the backend endpoints matched what Supabase expected — checking that REST calls for creating, listing, and deleting projects and summarizations all worked correctly against the live database.

Field Review

I researched the meeting-AI and transcription industry and wrote the field review section of the technical report. I covered the major existing tools — Otter.ai, Fireflies.ai, Zoom AI Companion, Fathom, and tl;dv — and analysed what they offer and what they consistently lack. Most produce a single undifferentiated list of bullet points; none separate decisions from notes. I cited sources from ACL Anthology, arXiv, and industry reports to ground the review in published research.

Labeled Transcripts

I contributed approximately 8 labeled transcripts to `eval_set.jsonl`, including excerpts from public city council meeting minutes — real, verbatim, and messy, with decisions both with and without clear owners.

What I Learned

Doing the field review made me understand the difference between what NLP research has achieved and what is actually deployed in real products. There is a large body of work on meeting summarization, action item extraction, and dialogue act classification going back years, yet mainstream commercial tools still produce simple undifferentiated bullet lists. The research-to-product gap is real and wider than I expected. I also developed a clearer understanding of the different NLP subtasks our system combines: automatic speech recognition (Whisper), abstractive summarization (the two-sentence summary), and named entity and relation extraction (pulling out task, owner, and deadline as structured fields). Each has its own failure modes and literature, and the overall system quality is limited by whichever stage performs worst on a given input.

Individual Reflection — Smaragda

My Contributions

Evaluation Set Design and Annotation Methodology

I designed the evaluation set the whole team used to measure the system. This included deciding on the format of `eval_set.jsonl`, writing the annotation guidelines the team followed when labeling, and coordinating labeling across all five members so everyone applied the same rules consistently.

What counts as an action item: A concrete commitment by a named or identifiable person — not a vague intention or a plan the team discussed but did not commit to.

Missing deadlines: Leave the deadline field blank rather than infer one.

Overlapping items: When one transcript has multiple overlapping action items for the same person, list each separately.

I also ensured the test set covered a realistic range of meeting types: standups, planning sessions, sales calls, retrospectives, and tricky cases (no action items at all, ambiguous ownership, hypothetical discussions). I contributed approximately 8 transcripts myself, focusing on multilingual scenarios.

Executive Summary

I wrote `deliverables/executive_summary.md` — the one-page non-technical description of the project. I led with the cost of unproductive meetings and the specific barrier multilingual teams face, then described what SummarAI does differently, then summarised the evaluation findings including the main limitation around owner attribution.

User Manual

I wrote `docs/user_manual.md` from scratch, walking through every feature: uploading a file, choosing the output language, reading the summary and takeaway bubbles, checking off action items, saving to a project, and browsing the project timeline. I also documented supported formats, known limitations, and common questions.

Slides

I put together the presentation slides (`deliverables/slides.md`), structuring the 20-minute presentation as: problem and motivation → field review → system overview → evaluation method and results → failure cases → future directions → Q&A.;

What I Learned

Building the annotation guidelines taught me that the definition of a label is itself a research decision with real consequences for the evaluation. We started with a vague shared understanding of what an action item was, and when I tried to write it down precisely, edge cases appeared immediately: is 'we should probably follow up on this' an action item? What if no one is named as responsible? What if someone agrees to do something only conditionally? These decisions determine what the model is

being evaluated against — so they determine what the evaluation actually measures. I now understand why inter-annotator agreement is reported in NLP papers: it is not a formality, it is evidence that the label definition is precise enough to be applied consistently. I also understood more deeply what the summarization task is about: not just shortening text, but identifying what is important and representing it in a structured, reusable form.